Email Spam Detection System

Complete Project Process • GitHub Documentation • Interview Preparation

Author: Sharon Darla | BCA Graduate | Aspiring Data Analyst

Project based on the resume: Python, Pandas, NLTK, Scikit-learn, TF-IDF, Multinomial Naive Bayes, train-test split, accuracy, precision and recall.

1. Project Overview

The Email Spam Detection System is a machine learning and Natural Language Processing (NLP) project. The purpose is to automatically classify an email as **Spam** or **Ham (legitimate)**. Because email messages are text, the project first converts raw text into clean numerical features and then uses a machine learning classifier to make predictions.

2. Problem Statement

Email users receive unwanted messages such as advertisements, scams, or suspicious content. Manually identifying every unwanted email is time-consuming. This project uses machine learning to learn patterns from previously labelled emails and predict whether a new message is spam or legitimate.

3. Project Objective

- Collect and load a labelled email dataset.
- Understand the dataset and check its structure.
- Clean and preprocess email text.
- Remove unnecessary characters and stopwords.
- Convert text into numerical features using TF-IDF.
- Split the data into training and testing sets.
- Train a Multinomial Naive Bayes classification model.
- Predict Spam/Ham for unseen emails.
- Evaluate the model using accuracy, precision and recall.

4. Technologies and Libraries

Technology / Library	Purpose
Python	Main programming language
Pandas	Load, clean and analyze tabular data
NumPy	Numerical operations where required
NLTK	Natural language text preprocessing
Scikit-learn	Vectorization, model training and evaluation
TF-IDF	Convert text into numerical features
Multinomial Naive Bayes	Classify emails into Spam/Ham
Google Colab / Jupyter	Development and experimentation

5. Complete Project Workflow

```
Dataset → Data Understanding → Data Cleaning → Text Preprocessing → Feature Extraction (TF-IDF) →
Train-Test Split → Model Training → Prediction → Evaluation → Final Spam/Ham Classification
```

6. Step-by-Step Implementation

Step 1 — Import Required Libraries

Import the libraries needed for data handling, text preprocessing, machine learning and evaluation.

```
import pandas as pd
import numpy as np
import nltk
import string
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, classification_report
```

Step 2 — Load the Dataset

Load the email dataset into a Pandas DataFrame. The dataset should contain the email text and its corresponding label, such as Spam or Ham.

```
df = pd.read_csv('email_dataset.csv')
print(df.head())
print(df.shape)
print(df.columns)
```

Step 3 — Understand the Data

Before training a model, inspect the rows, columns, missing values, duplicate records and label distribution. This helps identify data-quality issues before preprocessing.

```
print(df.info())
print(df.isnull().sum())
print(df.duplicated().sum())
print(df['label'].value_counts())
```

Step 4 — Handle Missing and Duplicate Data

Missing email text should be handled appropriately, and duplicate records can be removed so that repeated examples do not unnecessarily influence the model.

```
df = df.dropna(subset=['text'])
df = df.drop_duplicates()
```

Step 5 — Text Preprocessing

Raw email text contains punctuation, uppercase letters, stopwords and other information that may not be useful for the classifier. The text is cleaned before feature extraction.

Typical preprocessing steps in this project include:
• Convert text to lowercase
• Remove punctuation
• Remove stopwords using NLTK
• Keep the cleaned text for feature extraction

```
def clean_text(text):
    text = text.lower()
    text = text.translate(str.maketrans('', '', string.punctuation))
    words = text.split()
    # remove stopwords using an NLTK stopword list
    return ' '.join(words)
```

Step 6 — Separate Features and Target

The email text is the input feature, while the Spam/Ham label is the target that the model learns to predict.

```
X = df['text']
y = df['label']
```

Step 7 — Convert Text into Numerical Features Using TF-IDF

Machine learning algorithms cannot directly work with raw sentences. TF-IDF (Term Frequency–Inverse Document Frequency) converts each email into a numerical vector. It gives higher importance to words that are useful for distinguishing documents.

```
vectorizer = TfidfVectorizer()
X_tfidf = vectorizer.fit_transform(X)
```

Step 8 — Split Data into Training and Testing Sets

The dataset is divided into training and testing data. The training data is used to learn patterns, while the test data is used to check how well the model performs on unseen emails.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_tfidf, y, test_size=0.2, random_state=42
)
```

Step 9 — Train the Multinomial Naive Bayes Model

A Multinomial Naive Bayes classifier is trained on the TF-IDF features. It is commonly used for text classification because it can work effectively with word-based feature representations.

```
model = MultinomialNB()
model.fit(X_train, y_train)
```

Step 10 — Make Predictions

After training, the model predicts the labels of the test emails.

```
y_pred = model.predict(X_test)
```

Step 11 — Evaluate the Model

The project evaluates classification performance using accuracy, precision and recall. These metrics provide different views of model performance.

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, pos_label='spam')
recall = recall_score(y_test, y_pred, pos_label='spam')
print('Accuracy:', accuracy)
print('Precision:', precision)
print('Recall:', recall)
```

Important: The actual metric values should come from your own model run. Do not put an estimated accuracy in your GitHub README or interview answer.

7. Understanding the Evaluation Metrics

Metric	Simple meaning
Accuracy	Overall percentage of predictions that are correct.
Precision	Among emails predicted as spam, how many are actually spam.
Recall	Among actual spam emails, how many the model successfully identifies.

8. Predicting a New Email

For a new email, apply the same preprocessing and TF-IDF transformation used during training. Then pass the transformed text to the trained model.

```
new_email = ['Congratulations! You have won a prize.']
new_email_tfidf = vectorizer.transform(new_email)
prediction = model.predict(new_email_tfidf)
print(prediction)
```

The output can be interpreted as the predicted class, such as Spam or Ham, depending on the exact labels used in the dataset.

9. Challenges Faced and Solutions

Challenge	How it was handled
Noisy email text	Applied text cleaning and preprocessing.
Punctuation and unnecessary words	Removed punctuation and stopwords.
ML model cannot directly understand raw text	Used TF-IDF to create numerical features.
Need to test on unseen data	Used train-test split.
Need to measure model performance	Used accuracy, precision and recall.

10. What I Learned from the Project

- How to work with a real-world text dataset.
- How data cleaning affects machine learning.
- Basics of NLP and text preprocessing.
- How TF-IDF represents text numerically.
- How Multinomial Naive Bayes performs text classification.
- How to split data for training and testing.
- How to evaluate a classification model.
- How to explain a complete machine learning workflow.

11. Project Folder Structure for GitHub

```
Email-Spam-Detection-ML/
■■■ Email_Spam_Detection.ipynb
■■■ dataset/
■ ■■■ email_dataset.csv
■■■ README.md
■■■ requirements.txt
```

12. Suggested requirements.txt

```
pandas
numpy
nltk
scikit-learn
```

13. GitHub README Summary

This project demonstrates an end-to-end machine learning workflow for email spam classification. It covers data loading, data understanding, cleaning, NLP preprocessing, TF-IDF feature extraction, train-test splitting, Multinomial Naive Bayes training, prediction, and evaluation using accuracy, precision and recall.

14. Interview Explanation — 60 to 90 Seconds

“My project is an Email Spam Detection System using Python and Machine Learning. The main objective was to classify emails as spam or legitimate. I first loaded and analyzed the email dataset and handled data-quality issues such as missing and duplicate records. Then I performed text preprocessing by converting the text to lowercase and removing punctuation and stopwords using NLTK. After preprocessing, I used TF-IDF to convert the email text into numerical features. I split the data into training and testing sets and trained a Multinomial Naive Bayes classifier using Scikit-learn. Finally, I predicted the test data and evaluated the model using accuracy, precision, and recall. The main challenges were cleaning the text and converting text into numerical features. This project helped me understand the complete machine learning and NLP workflow.”

15. Short Answer: Why did you choose this project?

“I chose this project because email spam is a practical classification problem and it allowed me to learn both NLP and machine learning. It gave me hands-on experience in data preprocessing, feature extraction, model training, prediction, and evaluation.”

16. Future Improvements

- Compare Naive Bayes with Logistic Regression, Support Vector Machine and other suitable classifiers.
- Tune model and vectorizer parameters.
- Handle class imbalance if it exists in the dataset.
- Add a confusion matrix and other visualizations.
- Create a simple web interface for real-time prediction.
- Deploy the trained model as a web application.

17. Important Note for Your Resume and Interview

Keep your explanation consistent with the work you actually performed. If your original notebook used different column names, preprocessing steps, model settings, dataset name, or metrics, update this documentation to match your actual implementation. Especially, use your real model accuracy, precision and recall instead of example values.